

렌더링 속도를 개선하기

동명대학교 게임공학과

강영민

Camera 객체 만들기

```
class Camera :
    def __init__(self, eye=None, at=None, up=None) :
        # parameters for gluLookAt
        self.eye = eye if eye is not None else [0, 1, 3]
        self.at = at if at is not None else [0, 0, 0]
        self.up = up if up is not None else [0, 1, 0]

        # parameters for gluPerspective
        self.fovY = 60
        self.aspect = 1 # width/height
        self.near = 0.1
        self.far = 100

    def apply(self):
        glMatrixMode(GL_PROJECTION)
        glLoadIdentity()
        gluPerspective(self.fovY, self.aspect, self.near, self.far)

        glMatrixMode(GL_MODELVIEW)
        glLoadIdentity()
        gluLookAt(self.eye[0], self.eye[1], self.eye[2],
                 self.at[0], self.at[1], self.at[2],
                 self.up[0], self.up[1], self.up[2])

main_camera = Camera()
```

myglfw.py 개선

```
import glfw
```

```
#####
```

```
def initialize_window(w, h, window_title):
```

```
    if not glfw.init():
```

```
        return None
```

```
    window = glfw.create_window(w, h, window_title, None, None)
```

```
    if not window:
```

```
        glfw.terminate()
```

```
        print("window initialization failed")
```

```
        exit(0)
```

```
    glfw.make_context_current(window)
```

```
    return window
```

```
def main_loop(window, display_func):
```

```
    while not glfw.window_should_close(window):
```

```
        display_func(window)
```

```
        glfw.swap_buffers(window)
```

```
        glfw.poll_events()
```

```
    glfw.destroy_window(window)
```

```
    glfw.terminate()
```

```
    glfw.destroy_window(window)
```

```
    glfw.terminate()
```

```
def register_reshape(window, reshape_func):
```

```
    glfw.set_framebuffer_size_callback(window, reshape_func)
```

```
    # Manually apply initial reshape
```

```
    width, height = glfw.get_framebuffer_size(window)
```

```
    reshape_func(window, width, height)
```

```
def register_initGL(window, initialize_func):
```

```
    # Call initialize
```

```
    initialize_func(window)
```

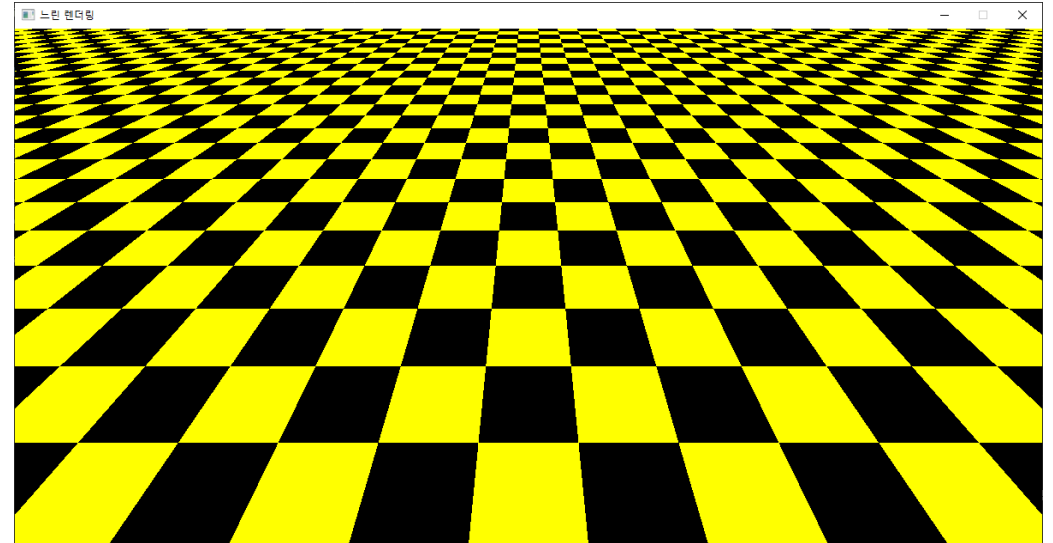
```
def register_keyboard(window, key_callback):
```

```
    glfw.set_key_callback(window, key_callback)
```

```
#####
```

정점이 많은 기하 객체

```
def drawPlane() :  
    n = 50 # 그려지는 바둑판 모양의 땅을 몇 개의 점으로 나눌 것인가  
    w = 50 # 그려지는 바둑판 모양의 땅이 너비  
    d = w / (n-1) # 각 줄 사이의 간격  
  
    startX = -w/2  
    startZ = -w/2  
  
    glColor3f(1, 1, 0)  
    glBegin(GL_QUADS)  
    for i in range(n):  
        for j in range(n):  
            if (i+j)%2 == 0 :  
                X = startX + i*d  
                Z = startZ + j*d  
                glVertex3f(X, 0, Z)  
                glVertex3f(X+d, 0, Z)  
                glVertex3f(X+d, 0, Z+d)  
                glVertex3f(X, 0, Z+d)  
    glEnd()
```



OpenGL codes

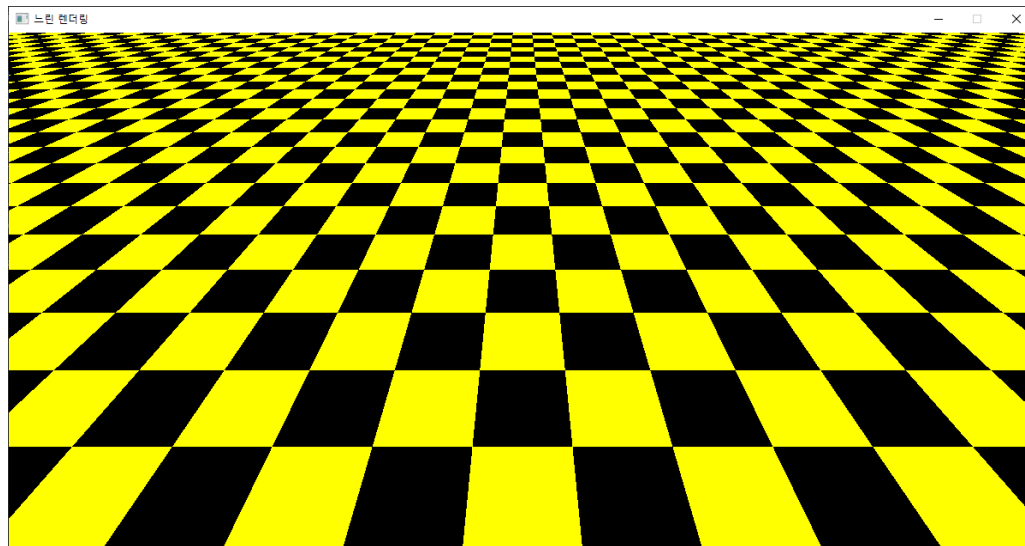
```
def initialize(window):  
    glClearColor(0.0, 0.0, 0.0, 1.0)
```

```
def display(window):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)  
    main_camera.apply()  
    drawPlane()
```

```
def reshape(window, width, height):  
    glViewport(0, 0, width, height)  
    main_camera.aspect = width / height  
    main_camera.apply()
```

실행

```
def main():  
    window = initialize_window(500, 500, "camera test")  
  
    register_initGL(window, initialize)  
    register_reshape(window, reshape)  
  
    # Main loop calling display  
    main_loop(window, display)  
  
if __name__ == "__main__":  
    main()
```



카메라를 움직여보자

```
def key_callback(window, key, scancode, action, mods):
    global main_camera
    if action == glfw.PRESS or action == glfw.REPEAT:
        if glfw.KEY_W == key :
            main_camera.eye[2] -= 0.1
            main_camera.at[2] -= 0.1
        elif glfw.KEY_S == key :
            main_camera.eye[2] += 0.1
            main_camera.at[2] += 0.1
        elif glfw.KEY_D == key :
            main_camera.eye[0] += 0.1
            main_camera.at[0] += 0.1
        elif glfw.KEY_A == key :
            main_camera.eye[0] -= 0.1
            main_camera.at[0] -= 0.1
```

```
def main():
    window = initialize_window(500, 500, "camera test")

    register_initGL(window, initialize)
    register_reshape(window, reshape)
    register_keyboard(window, key_callback)
```

...

정점의 수를 대폭 늘리기

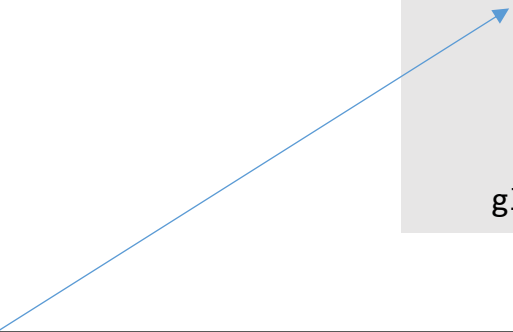
```
def drawPlane() :  
    n = 500      # 체스 판을 구성하는 한 면에 놓일 정점의 수  
    w = 100      # 체스 판의 한쪽 면 길이  
    d = w/(n-1)  # 인접한 두 정점 사이의 거리  
  
    # 체스판을 그린다  
    glBegin(GL_QUADS)  
    for i in range(n):  
        for j in range(n):  
            startX = -w/2 + i*d  
            startZ = -w/2 + j*d  
            # 행과 열의 번호 합이 짝수일 때만 그린다 (체스판)  
            if (i+j)%2 == 0:  
                glVertex3f(startX, 0, startZ)  
                glVertex3f(startX, 0, startZ+d)  
                glVertex3f(startX+d, 0, startZ+d)  
                glVertex3f(startX+d, 0, startZ)  
    glEnd()
```


매우 느린 이동

왜 이런 속도가 나타날까?

매번 CPU가 정점 지정을 하는 glVertex3f 호출이 반복됨

```
glBegin(GL_QUADS)
    for i in range(n):
        for j in range(n):
            startX = -w/2 + i*d
            startZ = -w/2 + j*d
            # 행과 열의 번호 합이 짝수일 때만 그린다 (체스판)
            if (i+j)%2 == 0:
                glVertex3f(startX, 0, startZ)
                glVertex3f(startX, 0, startZ+d)
                glVertex3f(startX+d, 0, startZ+d)
                glVertex3f(startX+d, 0, startZ)
glEnd()
```



```
def display(window):
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    main_camera.apply()
    drawPlane()
```

Display list 사용하기

- 리스트 만드는 방법

```
idxFirstList = glGenLists(nRange)
for i in range(nRange):
    glNewList(idxFirstList+i, GL_COMPILE)
    # 리스트를 구성하는 그리기 동작
    glEndList()
```

- nRange: 디스플레이 리스트의 개수
- glGenLists: nRange 개수의 리스트를 생성할 수 있도록 함
 - 연속된 번호 중 첫 번호를 리턴
- glNewList(리스트 번호, ...
 - 리스트 번호에 해당하는 리스트 정의를 시작
- glEndList
 - 만들고 있는 리스트를 완료함

리스트 호출

- 정의해 둔 리스트를 사용
 - 해당 그리기를 위해 준비한 리스트가 실행 됨
- 디스플레이 리스트가 빠른 이유
 - 명령 최적화
 - 렌더링 명령을 다시 하지 않고 이전에 정의된 명령을 다시 실행
 - 메모리 최적화
 - 상태 변경 감소
 - OpenGL 드라이버 최적화
- 주의점
 - 디스플레이 리스트는 동적인 데이터 렌더링에는 적합하지 않음

실제 사용예

```
def initialize(self):  
    self.drawPlaneList = glGenLists(1)  
    glColor3f(0,1,1)  
    glNewList(self.drawPlaneList, GL_COMPILE)  
    drawPlane()  
    glEndList()
```



실제 사용예

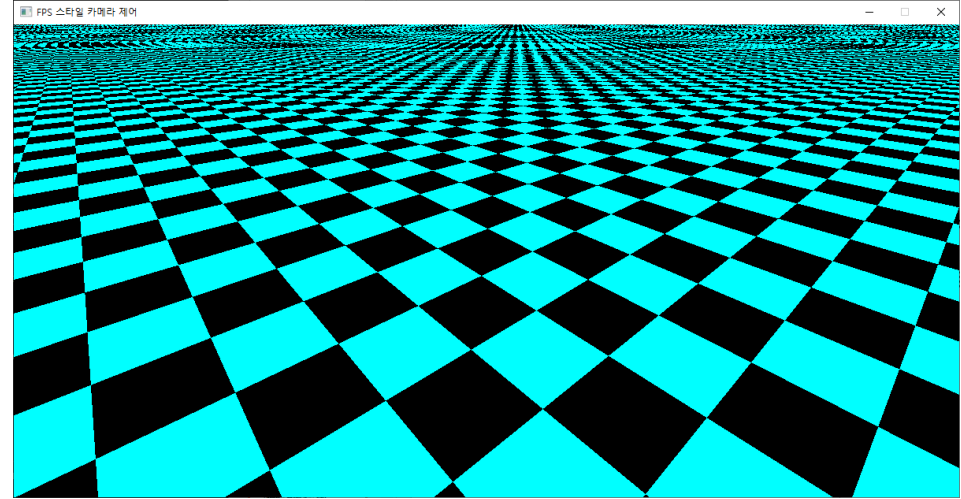
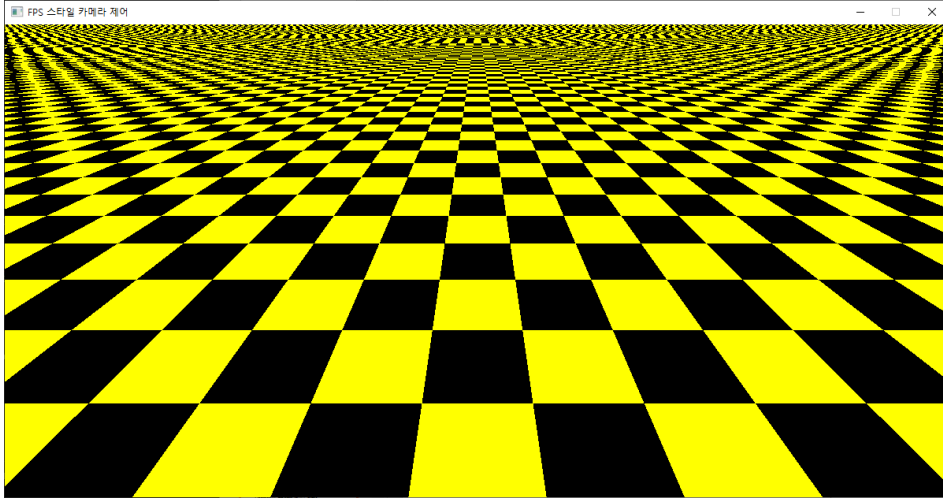
```
drawPlane_list = None
```

```
def initialize(window):  
    global drawPlane_list  
    glClearColor(0.0, 0.0, 0.0, 1.0)  
    drawPlane_list = glGenLists(1)  
    glColor3f(0,1,1)  
    glNewList(drawPlane_list, GL_COMPILE)  
    drawPlane()  
    glEndList()
```

```
def display(window):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)  
    main_camera.apply()  
    glCallList(drawPlane_list)
```

[실습] 키보드로 렌더링 방법 전환하여 속도 비교

- 키보드를 이용하여 평면을 그리는 방식을
 - Immediate 모드 $\leftrightarrow$ display list 모드로 전환하여
 - 카메라 이동의 속도를 비교해 보자
 - 그림을 그릴 때, 렌더링 모드에 따라 평면의 색이 다르게 하자



정점 배열 그리기

- `glDrawArrays`
 - 정점의 정보를 담고 있는 배열을 통째로 넘기는 방식
- 준비
 - `glEnableClientState(array)`
 - array: `GL_VERTEX_ARRAY`, `GL_COLOR_ARRAY`, `GL_NORMAL_ARRAY`
 - client vs Server in OpenGL
 - CPU – client
 - GPU – server
 - `glVertexArray(...)`
 - GPU가 접근해야 할 버퍼의 정보를 제공

parameters

glVertexPointer(size, type, stride, pointer)

size: 하나의 정점을 구성하는 좌표의 수. 2, 3, 또는 4 중에 하나이다.

type: 정점 배열에 들어 있는 숫자의 자료형으로 GL_INT, GL_FLOAT 등이 가능하다.

stride: 배열 내에서 정점들 사이의 간격을 바이트 단위로 표시한 것. 0이라면 사용하는 정점이 배열 내에서 간격이 0인 상태로 배치되어 있다는 의미이다.

pointer: 배열의 주소. C 언어의 경우 포인터 변수나 배열 이름이 되고, 파이썬에서는 리스트가 참조값으로 다루어지므로 리스트 변수를 그대로 사용하면 된다.

parameters

`glDrawArrays(primitive_mode, start_index, count)`

`primitive_mode`: 활성화된 정점 배열을 이용하여 그림을 그릴 때 어떤 방식으로 조합하여 그릴지를 결정하는 프리미티브 설정. GL_POINTS, GL_LINES 등이 가능하다.

`start_index`: 활성화된 정점 배열에서 그리기를 시작할 첫번째 정점의 인덱스

`count`: 사용할 정점의 총 개수

정점 버퍼 준비

- 배열, 리스트 등의 자료형에 담으면 됨

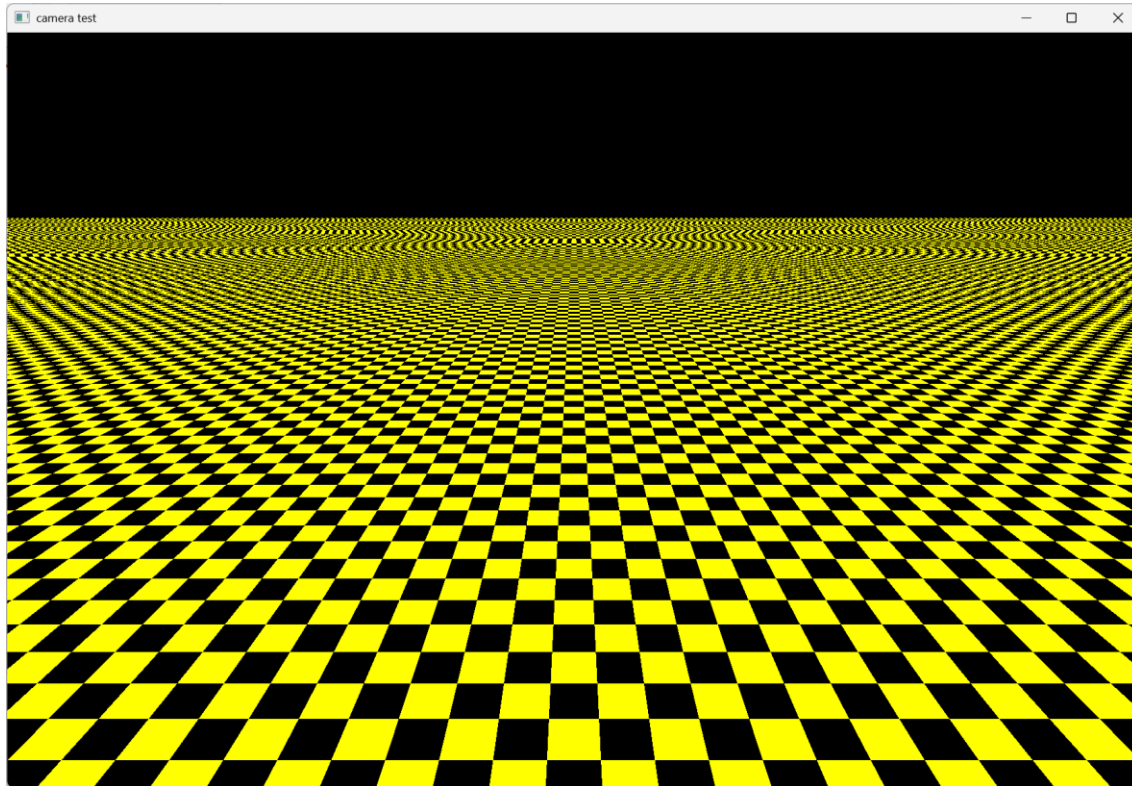
```
def drawPlane_buffers() :  
    n = 500 # 그려지는 바둑판 모양의 땅을 몇 개의 점으로 나눌 것인가  
    w = 50 # 그려지는 바둑판 모양의 땅이 너비  
    d = w / (n-1) # 각 줄 사이의 간격  
  
    v_buffer = []  
  
    startX = -w/2  
    startZ = -w/2  
  
    glColor3f(1, 1, 0)  
  
    for i in range(n):  
        for j in range(n):  
            if (i+j)%2 == 0 :  
                X = startX + i*d  
                Z = startZ + j*d  
                v_buffer.append([X, 0, Z])  
                v_buffer.append([X+d, 0, Z])  
                v_buffer.append([X+d, 0, Z+d])  
                v_buffer.append([X, 0, Z+d])  
  
    return v_buffer
```

정점 버퍼를 GPU에게 제공

```
def initialize(window):  
    global vertex_buffer  
    glClearColor(0.0, 0.0, 0.0, 1.0)  
  
    vertex_buffer = drawPlane_buffers()  
  
    ## 그래픽 처리에서 클라이언트는 CPU / 서버는 GPU  
    ## glEnable은 서버 상태를 변경하는 것  
    glEnableClientState(GL_VERTEX_ARRAY)          # 정점 버퍼 사용  
    glVertexPointer(3, GL_FLOAT, 0, vertex_buffer) # 정점 버퍼 설정  
  
    # 서버(GPU)는 지정된 정점 버퍼의 내용을 가져다가 사용하게 됨
```

버퍼를 이용한 그리기

```
def display(window):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)  
    main_camera.apply()  
    glDrawArrays(GL_QUADS, 0, len(vertex_buffer))
```



glDrawArrays를 이용한 렌더링을
동시에 적용한 결과

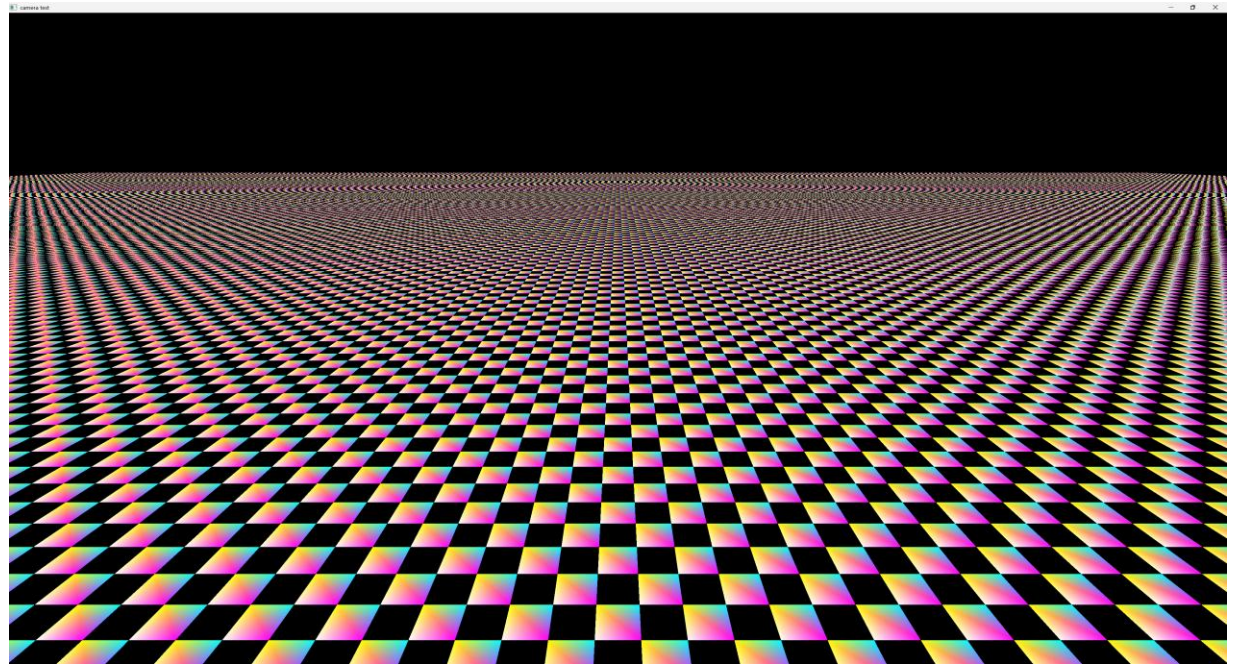
정점에 색상 추가

```
def drawPlane_buffers() :  
    n = 500 # 그려지는 바둑판 모양의 땅을 몇 개의 점으로 나눌 것인가  
    w = 50 # 그려지는 바둑판 모양의 땅이 너비  
    d = w / (n-1) # 각 줄 사이의 간격  
  
    v_buffer = []  
    c_buffer = []  
  
    startX = -w/2  
    startZ = -w/2  
  
    for i in range(n):  
        for j in range(n):  
            if (i+j)%2 == 0 :  
                X = startX + i*d  
                Z = startZ + j*d  
                v_buffer.append([X, 0, Z])  
                c_buffer.append([1,1,0])  
                v_buffer.append([X+d, 0, Z])  
                c_buffer.append([0,1,1])  
                v_buffer.append([X+d, 0, Z+d])  
                c_buffer.append([1,0,1])  
                v_buffer.append([X, 0, Z+d])  
                c_buffer.append([1,1,1])  
  
    return v_buffer, c_buffer
```

정점에 색상 추가

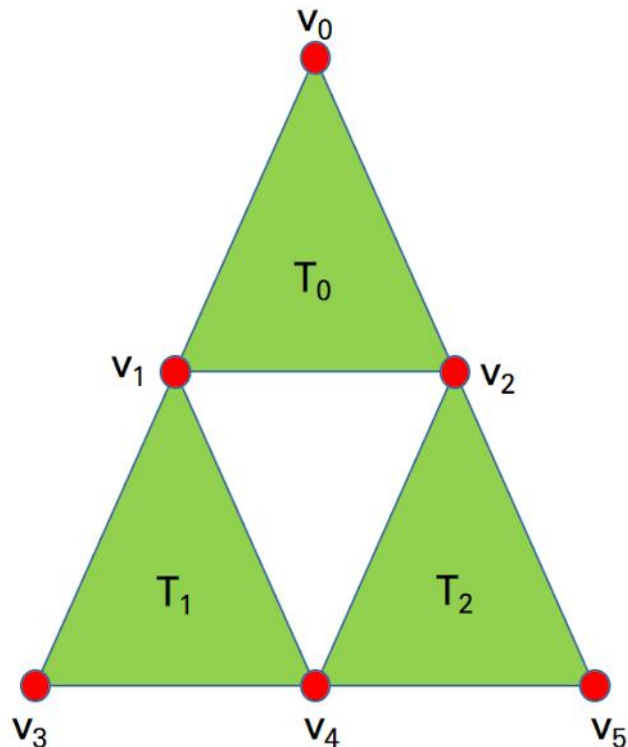
```
vertex_buffer = None  
color_buffer = None
```

```
def initialize(window):  
    global vertex_buffer, color_buffer  
    glClearColor(0.0, 0.0, 0.0, 1.0)  
    vertex_buffer, color_buffer = drawPlane_buffers()  
    glEnableClientState(GL_VERTEX_ARRAY)  
    glVertexPointer(3, GL_FLOAT, 0, vertex_buffer)  
    glEnableClientState(GL_COLOR_ARRAY)  
    glColorPointer(3, GL_FLOAT, 0, color_buffer)
```



정점 배열을 glDrawElements로 그리기

- glDrawArray 방식의 문제



```
vertex_array = [ [v0x, v0y, v0z ], [v1x, v1y, v1z ], [v2x, v2y, v2z ],  
                [v1x, v1y, v1z ], [v3x, v3y, v3z ], [v4x, v4y, v4z ],  
                [v2x, v2y, v2z ], [v4x, v4y, v4z ], [v5x, v5y, v5z ] ]
```

정점 배열 사용 가능하게 설정

```
glEnableClientState(GL_VERTEX_ARRAY)
```

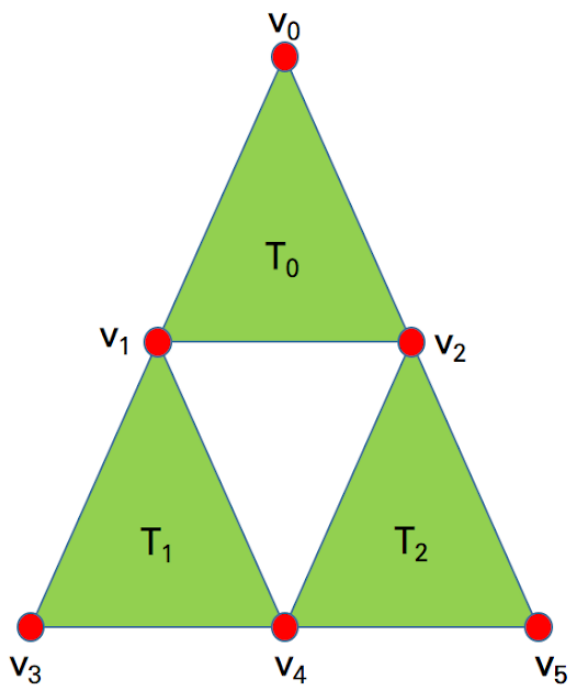
```
glVertexPointer(3, GL_FLOAT, 0, vertex_array)
```

정점 배열 그리기

```
glDrawArrays(GL_TRIANGLES, 0, 9)
```

glDrawElements로 개선

- Indexed Facet



```
vertex_array = [ [v0x, v0y, v0z ], [v1x, v1y, v1z ], [v2x, v2y, v2z ],  
                 [v3x, v3y, v3z ], [v4x, v4y, v4z ], [v5x, v5y, v5z ] ]
```

```
Index = [ 0, 1, 2, # T1  
         1, 3, 4, # T2  
         2, 4, 5] # T3
```

```
# 정점 배열 사용 가능하게 설정  
glEnableClientState(GL_VERTEX_ARRAY)  
glVertexPointer(3, GL_FLOAT, 0, vertex_array)
```

```
# 인덱스를 이용하여 그리기  
glDrawElements(GL_TRIANGLES, 9, GL_FLOAT, index)
```


glDrawElements: mesh 그리기에 유용

